

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ  
ВЫСШЕГО ОБРАЗОВАНИЯ

**«Национальный исследовательский университет ИТМО»**

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

**Отчет по блоку задач №3 (Тимус)**

**Выполнил:**

Студент группы Р3209

Евграфов А. А.

**Преподаватель:**

Косяков М. С.

Санкт-Петербург, 2026 г.

## Задачи из 3-его блока Тимуса

### Задача №1 «1067. Структура папок»

#### Описание

Дано  $N$  различных путей к папкам. Каждый путь состоит из имен папок, разделенных обратной косой чертой. Нужно восстановить общую структуру каталогов и вывести ее в виде дерева. Папки одного уровня должны идти в лексикографическом порядке, а каждая вложенная папка печатается с отступом на один пробел больше, чем ее родитель.

**Ограничение времени:** 2 секунды.

**Ограничение памяти:** 64 МБ.

#### Решение

Будем строить дерево путей. Корень дерева соответствует пустой вершине, которая сама не выводится. Для каждой строки разбиваем путь по символу ‘\’ и последовательно спускаемся по дереву. Если очередной папки среди детей текущей вершины нет, создаем новую вершину.

Чтобы сразу поддерживать требуемый порядок вывода, детей каждой вершины храним в `map<string, int>`. После добавления всех путей запускаем обход в глубину от корня: печатаем имя вершины с отступом, равным глубине, и рекурсивно печатаем ее детей.

#### Доказательство

При добавлении одного пути алгоритм создает ровно те вершины, которых еще не было в дереве, и переиспользует уже существующие вершины для общего префикса пути. Поэтому после обработки всех строк в дереве существует вершина для каждой папки, встречающейся хотя бы в одном входном пути, и ребро между папкой и ее непосредственной подпапкой.

Обход в глубину выводит каждую папку сразу после ее родительской папки, увеличивая глубину на единицу при переходе к ребенку. Значит, количество пробелов перед именем совпадает с глубиной папки в иерархии. Так как дети каждой вершины хранятся в `map`, они перебираются в лексикографическом порядке.

#### Алгоритмическая сложность

- **По времени:**  $O(S \log S)$ , где  $S$  — суммарное количество имен папок во всех путях. Логарифм появляется из-за вставок в `map`.
- **По памяти:**  $O(S)$ , так как каждая различная папка хранится в дереве один раз.

## Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 struct Node {
6     map<string, int> children;
7 };
8
9 vector<Node> tree(1);
10
11 void addPath(const string& path) {
12     int vertex = 0;
13     string name;
14
15     for (size_t i = 0; i <= path.size(); i++) {
16         if (i == path.size() || path[i] == '\\') {
17             if (!tree[vertex].children.count(name)) {
18                 tree[vertex].children[name] = static_cast<int>(tree.size());
19                 tree.push_back(Node());
20             }
21             vertex = tree[vertex].children[name];
22             name.clear();
23         } else {
24             name += path[i];
25         }
26     }
27 }
28
29 void printTree(int vertex, int depth) {
30     for (const auto& child : tree[vertex].children) {
31         cout << string(depth, ' ') << child.first << '\n';
32         printTree(child.second, depth + 1);
33     }
34 }
35
36 int main() {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     int n;
41     cin >> n;
42
43     string path;
44     for (int i = 0; i < n; i++) {
45         cin >> path;
46         addPath(path);
47     }
48
49     printTree(0, 0);
50     return 0;
51 }
```

## Задача №2 «Белые полосы»

### Описание

Дана таблица размера  $m \times n$ . Некоторые клетки закрашены в черный цвет, остальные являются белыми. Белой полосой называется максимальный по включению белый отрезок размера  $1 \times l$  или  $l \times 1$ . Нужно найти количество белых полос.

**Ограничение времени:** 1 секунда.

**Ограничение памяти:** 64 МБ.

### Решение

Отрезки длины хотя бы 2 считаем отдельно по строкам и столбцам. Рассмотрим одну строку: отсортированные черные клетки делят ее на белые промежутки, и каждый промежуток длины хотя бы 2 является горизонтальной белой полосой. Аналогично считаем вертикальные полосы по столбцам.

Остается учесть полосы длины 1. Одиночная белая клетка является максимальной полосой только тогда, когда у нее нет белых соседей по стороне, то есть все существующие соседние клетки черные. Такие клетки ищем среди соседей черных клеток; дополнительно обрабатываем случай таблицы  $1 \times 1$  без черных клеток.

### Доказательство

В фиксированной строке черные клетки делят все белые клетки на независимые непрерывные промежутки. Каждый промежуток длины хотя бы 2 не может быть расширен влево или вправо. Так как его длина больше единицы, он не может быть частью вертикального отрезка размера  $l \times 1$ . Поэтому каждый такой промежуток является максимальной горизонтальной белой полосой. Аналогично для вертикальных промежутков длины хотя бы 2.

Белая полоса длины 1 состоит из одной клетки. Она максимальна тогда и только тогда, когда ее нельзя расширить ни горизонтально, ни вертикально, то есть рядом по стороне нет белых клеток. Проверка всех кандидатов среди соседей черных клеток находит все такие клетки: если таблица больше  $1 \times 1$ , у изолированной белой клетки обязательно есть хотя бы один сосед-черная клетка. Сумма длинных горизонтальных полос, длинных вертикальных полос и изолированных белых клеток это ответ.

### Алгоритмическая сложность

- По времени:  $O(k \log k)$ , где  $k$  — количество черных клеток.
- По памяти:  $O(k)$

## Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 long long encode(int x, int y, int n) {
6     return 1LL * x * (n + 1) + y;
7 }
8
9 long long countStripes(map<int, vector<int>>& lines, int cnt, int len) {
10     long long ans = 0;
11
12     if (len >= 2) {
13         ans += cnt - (int)lines.size();
14     }
15
16     for (auto& it : lines) {
17         vector<int>& v = it.second;
18         sort(v.begin(), v.end());
19
20         int stripes = 0;
21         int last = 0;
22
23         for (int x : v) {
24             if (x - last - 1 >= 2) {
25                 stripes++;
26             }
27             last = x;
28         }
29
30         if (len - last >= 2) {
31             stripes++;
32         }
33
34         ans += stripes;
35     }
36
37     return ans;
38 }
39
40 int main() {
41     ios::sync_with_stdio(false);
42     cin.tie(nullptr);
43
44     int m, n, k;
45     cin >> m >> n >> k;
46
47     map<int, vector<int>> rows;
48     map<int, vector<int>> cols;
49     unordered_set<long long> black;
50     unordered_set<long long> cand;
51
52     int dx[] = {-1, 1, 0, 0};
53     int dy[] = {0, 0, -1, 1};
54
55     for (int i = 0; i < k; ++i) {
56         int x, y;
57         cin >> x >> y;
58         rows[x].push_back(y);
```

```

59     cols[y].push_back(x);
60     black.insert(encode(x, y, n));
61 }
62
63 for (auto code : black) {
64     int x = static_cast<int>(code / (n + 1));
65     int y = static_cast<int>(code % (n + 1));
66
67     for (int d = 0; d < 4; ++d) {
68         int nx = x + dx[d];
69         int ny = y + dy[d];
70
71         if (nx < 1 || nx > m || ny < 1 || ny > n) {
72             continue;
73         }
74
75         long long cur = encode(nx, ny, n);
76         if (!black.count(cur)) {
77             cand.insert(cur);
78         }
79     }
80 }
81
82 if (m == 1 && n == 1 && k == 0) {
83     cand.insert(encode(1, 1, n));
84 }
85
86 long long single = 0;
87 for (auto code : cand) {
88     int x = static_cast<int>(code / (n + 1));
89     int y = static_cast<int>(code % (n + 1));
90     bool ok = true;
91
92     for (int d = 0; d < 4; ++d) {
93         int nx = x + dx[d];
94         int ny = y + dy[d];
95
96         if (nx < 1 || nx > m || ny < 1 || ny > n) {
97             continue;
98         }
99
100        if (!black.count(encode(nx, ny, n))) {
101            ok = false;
102            break;
103        }
104    }
105
106    if (ok) {
107        single++;
108    }
109 }
110
111 long long ans = 0;
112 ans += countStripes(rows, m, n);
113 ans += countStripes(cols, n, m);
114 ans += single;
115
116 cout << ans << '\n';
117 return 0;
118 }

```